

1. Introduction to AI

Definition: Artificial Intelligence (AI) is the field of computer science focused on building smart machines capable of performing tasks that typically require human intelligence.

Usage:

- Chatbots
- Personal assistants (e.g., Siri, Alexa)
- Recommendation systems (e.g., Netflix, Amazon)

Advantages:

- Automates repetitive tasks
- Makes data-driven decisions
- Enhances customer experience

Disadvantages:

- High implementation cost
- Job displacement
- Dependence on data

2. ANN (Artificial Neural Network) & FNN (Feedforward Neural Network)

Definition: ANNs are computing systems inspired by biological neural networks. FNN is a type of ANN where connections between the nodes do not form a cycle.

Usage:

- Classification
- Regression
- Image recognition

Advantages:

- Learns complex patterns
- Supports parallel processing

Disadvantages:

- Requires large data
- Hard to interpret

Steps:

1. Input data
2. Forward pass through layers

3. Loss calculation
4. Backpropagation
5. Optimization

Syntax (Keras):

```
model = Sequential([
    Dense(32, activation='relu', input_shape=(input_dim,)),
    Dense(1, activation='sigmoid')
])
```

3. Dropout Layer

Definition: A regularization technique that randomly drops neurons during training to prevent overfitting.

Usage:

- Applied between Dense layers

Advantages:

- Prevents overfitting
- Improves generalization

Syntax:

```
from keras.layers import Dropout
model.add(Dropout(0.5))
```

4. Imbalance in Dataset

Definition: Occurs when classes in a dataset are not equally represented.

Techniques:

- Under-sampling
- Over-sampling
- SMOTE
- Ensemble methods

Problems:

- Biased predictions

Solutions:

```
from imblearn.over_sampling import SMOTE
X_res, y_res = SMOTE().fit_resample(X, y)
```

5. CNN (Convolutional Neural Network)

Definition: Specialized neural networks for processing data with grid-like topology (e.g., images).

Components:

- Convolutional layers
- Pooling layers
- Fully connected layers

Usage:

- Image classification
- Object detection

Advantages:

- Reduces parameters
- Extracts spatial features

Syntax:

```
model = Sequential([
    Conv2D(32, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(1, activation='sigmoid')
])
```

6. Activation Functions & Loss Functions

Common Activation Functions:

- ReLU: $\max(0, x)$
- Sigmoid: $1 / (1 + e^{-x})$
- Softmax: Used for multi-class classification

Common Loss Functions:

- Binary Crossentropy
- Categorical Crossentropy
- MSE (Mean Squared Error)

Usage:

- Activation: Introduces non-linearity
- Loss: Measures model error

7. Data Augmentation

Definition: Technique to increase dataset size by modifying existing data (e.g., flipping, rotating images).

Advantages:

- Prevents overfitting
- Improves performance

Syntax:

```
from keras.preprocessing.image import ImageDataGenerator
aug = ImageDataGenerator(rotation_range=40, horizontal_flip=True)
```

8. Transfer Learning

Definition: Uses pre-trained models for new tasks by re-training final layers.

Usage:

- When data is limited
- For complex models (e.g., VGG, ResNet)

Advantages:

- Faster training
- Requires less data

Syntax:

```
base_model = VGG16(include_top=False, input_shape=(224,224,3))
for layer in base_model.layers:
    layer.trainable = False
```

9. YOLO (You Only Look Once)

Definition: Real-time object detection algorithm

Steps:

1. Load model
2. Predict on image/video
3. Extract bounding boxes & labels

Usage:

- Surveillance
- Autonomous vehicles

Syntax:

```
model = YOLO("yolov8n.pt")
results = model.predict(source="video.mp4")
```

10. RNN (Recurrent Neural Network)

Definition: Neural networks with loops, useful for sequential data

Applications:

- Text classification
- Sentiment analysis

Limitations:

- Vanishing gradient

Syntax:

```
model = Sequential([
    Embedding(input_dim=1000, output_dim=64),
    SimpleRNN(128),
    Dense(1, activation='sigmoid')
])
```

11. Pipelines

Definition: A tool to automate machine learning workflow (preprocessing to prediction)

Steps:

1. Missing value handling
2. Feature encoding
3. Scaling
4. Model training

Syntax:

```
from sklearn.pipeline import Pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
```

12. BERT (Bidirectional Encoder Representations from Transformers)

Definition: A transformer-based model for NLP tasks

Usage:

- Text classification
- Named Entity Recognition

Advantages:

- Deep understanding of context

13. Streamlit

Definition: A Python library to build interactive web apps for ML models

Usage:

- Create dashboards
- Deploy models

Syntax:

```
import streamlit as st
st.title("My AI App")
st.file_uploader("Upload File")
```